Birla Institute of Technology and Science-Pilani, Hyderabad Campus
First Semester, 2025-26
BITS F232 Foundations of Data Structures and Algorithms
Mid Semester Examination (Closed Book)

Max. Marks: 50 (25%) **Time: 90 mins**

Q 1 [8 Marks] You are given a singly linked list containing n nodes. Without explicitly counting the nodes or performing any arithmetic operations (such as addition, subtraction, multiplication, or division), design an algorithm (give pseudo-code) that runs in $O(n)$ time to determine whether the number of nodes in the list is a power of two; that is, whether $n = 2^k$ for some integer $k \geq 0$.

Your algorithm should:

- Not use any auxiliary storage (such as arrays or additional lists),
- Not traverse the list multiple times solely for counting the nodes,
- Use only $O(1)$ extra space,
- Rely solely on pointer traversal without performing arithmetic operations on node indices or counters.

Additionally, provide a justification for the running time of your algorithm.

Solution:

Key Idea: We can exploit the property that in a list whose length is a power of two, repeatedly moving one pointer twice as fast as another will cause the faster pointer to reach the end *exactly* when the slower pointer reaches the end of a sublist of the same length.
Equivalently, we can repeatedly check whether the list can be evenly divided into two equal halves, without counting nodes. This can be done using the classic *fast and slow pointer* (tortoise and hare) technique.

Algorithm Outline:

1. Let `head` be the pointer to the first node of the list.
2. Repeatedly check whether the list can be divided into two halves by moving:
   - `slow` pointer: moves one step at a time.
   - `fast` pointer: moves two steps at a time.
3. If `fast` reaches the end exactly (`NULL`) after an integer number of such halvings, then the list length is a power of two.
4. If `fast` becomes `NULL` at any intermediate step in the middle of a traversal (i.e., cannot move exactly twice per slow move), then the length is not a power of two.

Pseudocode:

Algorithm 1 IsPowerOfTwo(`head`)

```
1: if head = NULL then
2:     return False                          ▷ Empty list (n = 0) is not a power of two
3: end if
4: curr ← head
5: while curr.next ≠ NULL do
6:     slow ← curr
7:     fast ← curr
8:     while fast.next ≠ NULL and fast.next.next ≠ NULL do
9:         slow ← slow.next
10:        fast ← fast.next.next
11:    end while
12:    if fast.next = NULL then
13:        return False                      ▷ Odd number of nodes in this round
14:    end if
15:    curr ← slow.next                      ▷ Proceed to next half
16: end while
17: return True
```

Marking Scheme: inner loop will carry 2 marks, outer loop 2 marks, and lines 1-4 carries 2 marks if the rest of the algorithm is correct.

Time Complexity: Each node is visited at most a constant number of times across all stages, so the total running time is $O(n)$. More precisely, the number of nodes visited in each round are $n/2, n/4, n/8, \ldots, 1$ and each node takes $O(1)$ time. So total time $= n/2 + n/4 + n/8 + \ldots + 1 = 2n = O(n)$.

Space Complexity: The algorithm uses only a constant number of pointer variables (`curr`, `slow`, `fast`), hence $O(1)$ extra space.
Marking Scheme: 2 Marks for time and space, if proper justification is provided.

Note: if you have used counters to count the number of nodes, you get 0 marks.

Q 2 [8 Marks] Suppose we need to maintain two stacks of the same type of items in a program. If the two stacks are stored in separate arrays, then one stack might overflow while there is considerable unused space in the other. To avoid this situation, it is preferable to maintain both stacks within the same array.

Explain how to implement two stacks in a single array $S[1..n]$ (the array indices start from 1) such that neither stack overflows unless the total number of elements in both stacks together equals n.

Write pseudocode for the following operations, ensuring that each operation runs in $O(1)$ time:

- **PUSHA**: Push an element onto Stack A
- **PUSHB**: Push an element onto Stack B
- **POPA**: Pop an element from Stack A
- **POPB**: Pop an element from Stack B

The variables `topA` and `topB` denote the indices of the top elements of Stack A and Stack B, respectively.

Solution:

Idea (3 Marks): To utilize the array space efficiently and avoid overflow unless the array is full, we let the two stacks grow toward each other:

- **Stack A** grows from the *left end* of the array (index 1) toward higher indices.

- **Stack B** grows from the *right end* of the array (index n) toward lower indices.

Initially,

$$\texttt{topA} = 0, \quad \texttt{topB} = n + 1.$$

`topA` always points to the top element of Stack A (0 means empty), and `topB` points to the top of Stack B ($n + 1$ means empty).

Overflow condition: An overflow occurs only when both stacks together occupy the entire array, i.e.

$$\texttt{topA + 1 = topB}.$$

Thus, neither stack overflows unless the total number of elements equals n.

Stack Operations (Pseudocode) (5 Marks):

Algorithm 2 PUSHA(x)

```
if topA + 1 = topB then
    error: Stack Overflow
else
    topA ← topA + 1
    S[topA] ← x
end if
```

Algorithm 3 PUSHB(x)

```
if topB - 1 = topA then
    error: Stack Overflow
else
    topB ← topB − 1
    S[topB] ← x
end if
```

Algorithm 4 POPA()

```
if topA = 0 then
    error: Stack A Underflow
else
    x ← S[topA]
    topA ← topA − 1
    return x
end if
```

Algorithm 5 POPB()

```
if topB = n + 1 then
    error: Stack B Underflow
else
    x ← S[topB]
    topB ← topB + 1
    return x
end if
```

Time Complexity: Each operation (`PUSHA`, `PUSHB`, `POPA`, `POPB`) performs only a constant number of assignments and comparisons, so it runs in $O(1)$ time.

Remark: This design ensures optimal use of the array. Neither stack overflows unless the total number of elements stored in both stacks together equals n.

Q 3 [8 Marks] You are given a Binary Search Tree (BST) containing n keys, and two integer keys k_1 and k_2 such that $k_1 \leq k_2$. Let h denote the height of the BST, and let m denote the number of keys in the BST that lie within the range $[k_1, k_2]$, inclusive.

Design an algorithm (write pseudo-code) that runs in $O(h + m)$ time to display all node values that lie within the range $[k_1, k_2]$, inclusive. The output should be displayed in sorted (ascending) order.

Hint: A standard in-order traversal visits all n nodes and takes $O(n)$ time. Consider how you might use the ordering property of Binary Search Trees (BSTs) to avoid visiting unnecessary parts of the tree when searching for keys within the range $[k_1, k_2]$.

Solution: **Idea:** A standard in-order traversal visits all n nodes and takes $O(n)$ time. However, we can use the BST property to skip unnecessary subtrees:

- If the current node's key is greater than k_1, then its left subtree may contain keys in the range, so we explore it.
- If the current node's key lies within the range, we print it.

- If the current node's key is less than k_2, then its right subtree may contain keys in the range, so we explore it.

This ensures that only the relevant portions of the tree are visited.

Algorithm:

Algorithm 6 RangeSearchBST($node, k_1, k_2$)

```
1: if node = NULL then
2:     return
3: end if
4: if node → key > k1 then
5:     RangeSearchBST(node→ left, k1, k2)
6: end if
7: if k1 ≤ node → key ≤ k2 then
8:     print(node.key)
9: end if
10: if node → key < k2 then
11:     RangeSearchBST(node→right, k1, k2)
12: end if
```

Time Complexity Analysis:

- The recursive search paths from the root to the first and last keys in the range may each involve at most $O(h)$ nodes.
- Exactly m nodes are printed (one per key in the range).

Therefore, the total running time is

$$T(n) = O(h + m).$$

Marking Scheme: there are 4 blocks (lines 1-3, 4-6, 7-9, and 10-12). Each block carries 2 marks.

Q 4 [8 Marks] You are given an array $A[1 \ldots n]$ of n distinct integers and a positive integer k (where $1 \leq k \leq n$). Design an algorithm that outputs the k smallest integers from A in $O(n + k \log n)$ time.

Write the complete pseudocode of your algorithm (even if you are using functions discussed in the class), and justify the running time of your solution.

Idea:

- Construct a *min-heap* from all n elements of A in $O(n)$ time using the bottom-up heapify procedure.
- Then, perform `DeleteMin()` operation k times to extract the k smallest elements.
- Each `DeleteMin()` takes $O(\log n)$ time, so total cost for k extractions is $O(k \log n)$.

Total running time: $O(n + k \log n)$

Algorithm (Pseudocode):

Algorithm 7 K_Smallest($A[1..n]$, k) 2 Marks

```
BuildMinHeap(A)                          ▷ O(n)
result ← empty list
for i ← 1 to k do
    x ← DeleteMin(A)                     ▷ O(log n)
    print(x)
end for
return result
```

Algorithm 8 HeapifyDown(A, i, n) 2 Marks

```
left ← 2i, right ← 2i + 1, smallest ← i
if left ≤ n and A[left] < A[smallest] then
    smallest ← left
end if
if right ≤ n and A[right] < A[smallest] then
    smallest ← right
end if
if smallest ≠ i then
    swap(A[i], A[smallest])
    HeapifyDown(A, smallest, n)
end if
```

Algorithm 9 DeleteMin(A) 1 Mark

```
n ← length(A)
min ← A[1]
A[1] ← A[n]
remove last element from A
HeapifyDown(A, 1, n − 1)
return min
```

Algorithm 10 BuildMinHeap(A) 2 Marks

```
n ← length(A)
for i ← ⌊n/2⌋ downto 1 do
    HeapifyDown(A, i, n)
end for
```

Time Complexity Analysis: 1 Mark

Operation	Time Complexity
BuildMinHeap	$O(n)$
k DeleteMin operations	$O(k \log n)$
Total	$O(n + k \log n)$

Q 5 [8 Marks] Given a sorted array $A[1 \dots n]$ of n distinct elements and a target element x, design an algorithm to determine whether x is present in the array.

Your algorithm must follow the *divide-and-conquer* paradigm by dividing the array into three almost equal parts and recursively searching in the appropriate segment, similar to how the binary search algorithm divides the array into two parts at each step. Write the complete pseudocode of your algorithm.

Further, write the recurrence relation that represents the running time of your algorithm and solve it using the method discussed in class.

Solution:

Pseudocode:

Algorithm 11 TernarySearch(A, l, r, x) Marking scheme: 5 Marks

```
if l > r then
    return False                                   ▷ Element not found
end if
third ← ⌊(r − l)/3⌋
mid1 ← l + third
mid2 ← r − third
if A[mid1] = x or A[mid2] = x then
    return True
else if x < A[mid1] then
    return TernarySearch(A, l, mid1 − 1, x)
else if x > A[mid2] then
    return TernarySearch(A, mid2 + 1, r, x)
else
    return TernarySearch(A, mid1 + 1, mid2 − 1, x)
end if
```

Initial Call:

$$\text{TernarySearch}(A, 1, n, x)$$

Recurrence Relation:

At each step, the array of size n is divided into three almost equal parts, and we recurse into only **one part**. The recurrence for running time $T(n)$ is:

$$T(n) = T(n/3) + O(1), \quad T(1) = O(1)$$

where $O(1)$ accounts for computing midpoints and comparisons.

Solving the Recurrence:

Unfolding the recurrence:

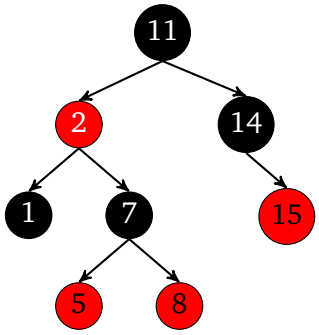
$$\begin{aligned} T(n) &= T(n/3) + c \\ &= T(n/3^2) + c + c \\ &= \dots \\ &= T(n/3^k) + k \cdot c \end{aligned}$$

We stop when $n/3^k = 1 \implies 3^k = n \implies k = \log_3 n$.

$$T(n) = O(\log_3 n) = O(\log n)$$

Marking scheme: 3 Marks for writing recurrence relation and solving it.

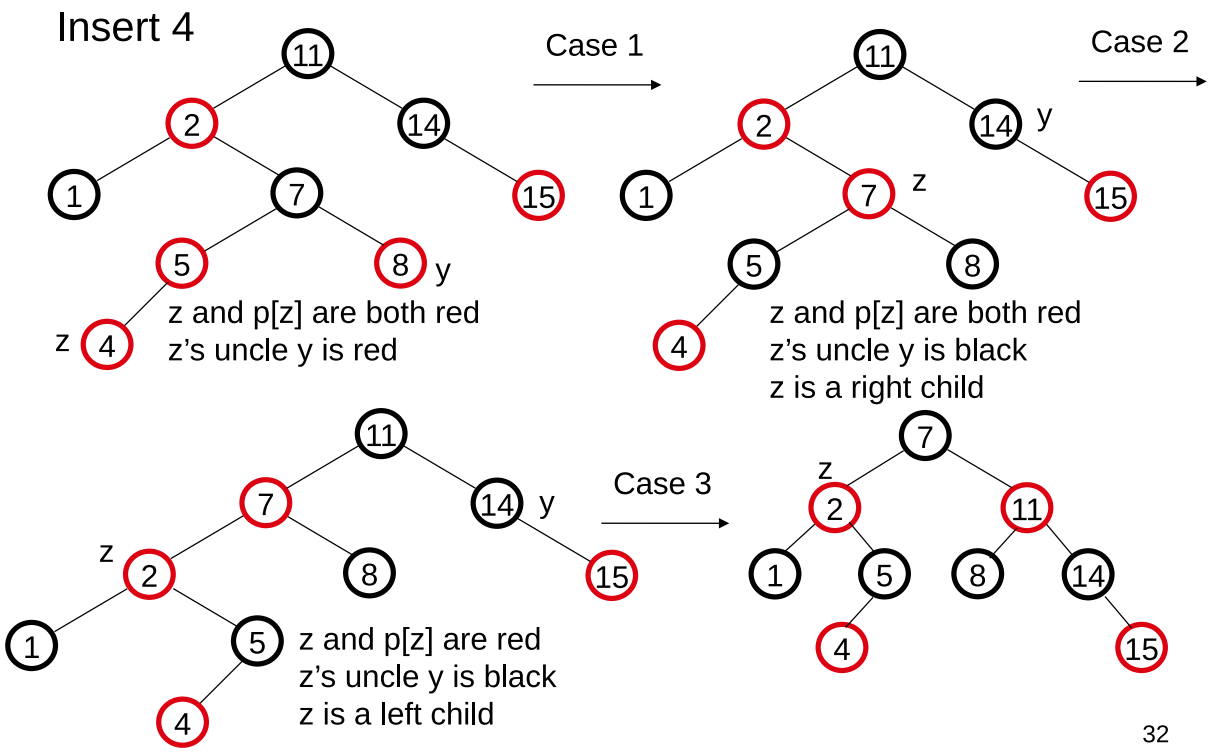
Q 6 (a) **[5 Marks]** Consider the following red-black tree with red nodes $\{2, 5, 8, 15\}$, and the remaining nodes are black. The NIL nodes are not shown in the figure.



Insert the key 4 into the above tree, and ensure that the resulting tree satisfies the red-black tree properties.

Clearly explain all the steps involved, including any recoloring and rotations performed, to maintain the red-black properties.

Solution:

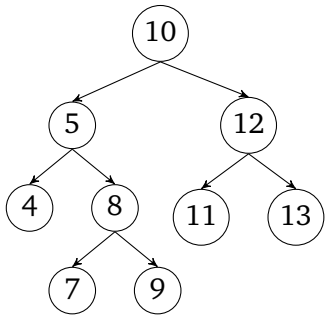


Marking scheme: 1 mark for insertion of 4 along with red color. 2 Marks for case 1, 1 mark each for case 2 and case 3

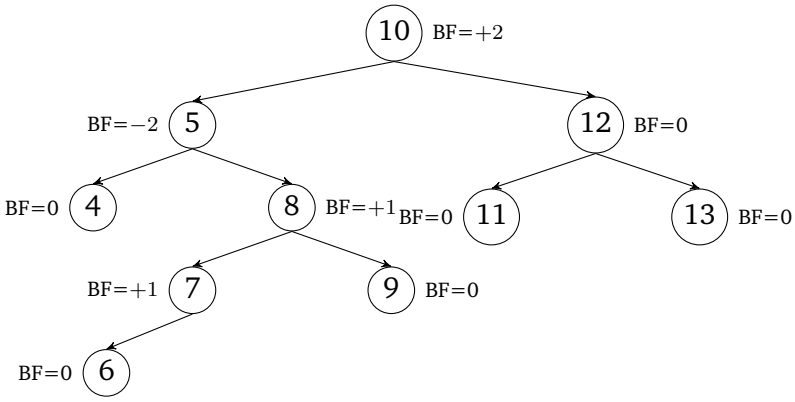
(b) **[5 Marks]** Consider the following AVL tree (i.e., a height-balanced binary search tree).

- Insert the key 6 into the tree.
- Check whether the resulting tree satisfies the AVL balance property.
- If the tree becomes unbalanced, perform the appropriate AVL rotation(s) to restore the balance and obtain a valid AVL tree.

Clearly show all your steps, including any rotations performed, to illustrate how the AVL property is maintained.

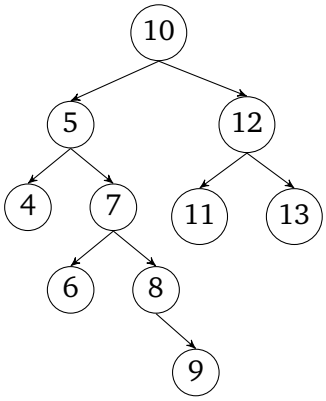


Solution:
The tree after insertion of 6 (before any rotations) is:

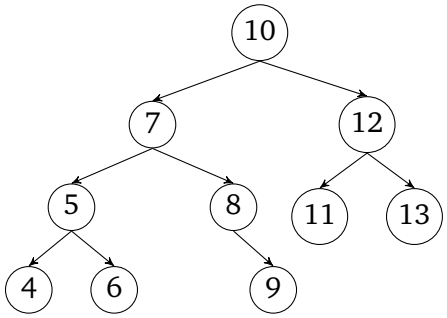


The pivot node is 5, and it is Right-Left heavy.

So, first do a right rotation at node 8 and the resultant tree is



Now do a left rotation at 5. The resultant tree is



Marking scheme: 1 Mark for inserting 6 correctly. 2 Marks for finding Balance factors, 1 mark each for two rotations.